

High-Performance Data Analysis

(Análise de Dados de Elevado Desempenho)

Distributed Machine Learning (Inference)

University of Minho

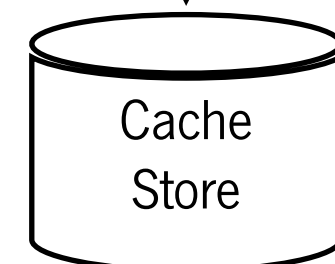
2025-2026



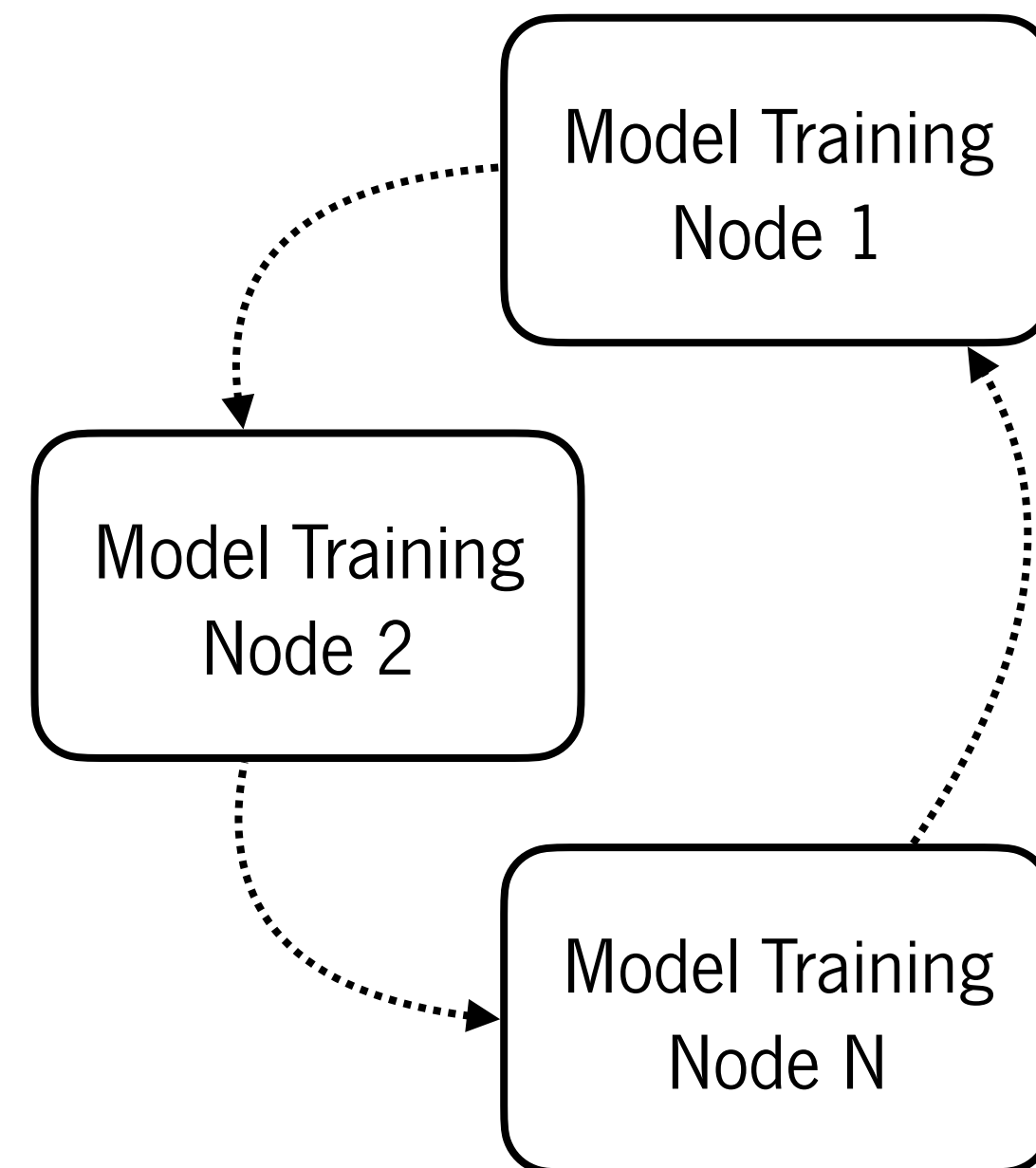
Closing The Loop: The Full ML Pipeline

Data processing (Apache Spark)

**Data ingestion /
preprocessing**



Distributed Training (DDP / DeepSpeed / Horovod)

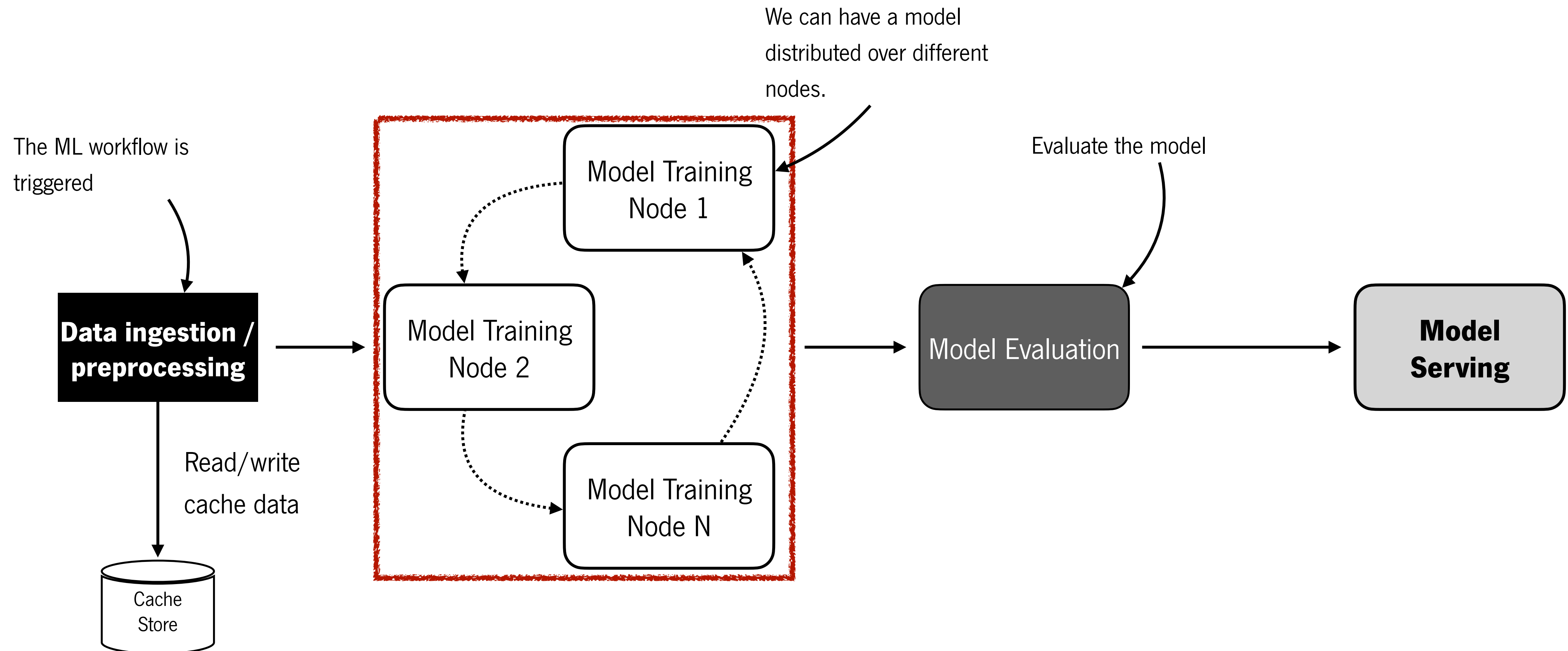


Distributed Serving (Ray / vLLM / TensorRT-LLM)

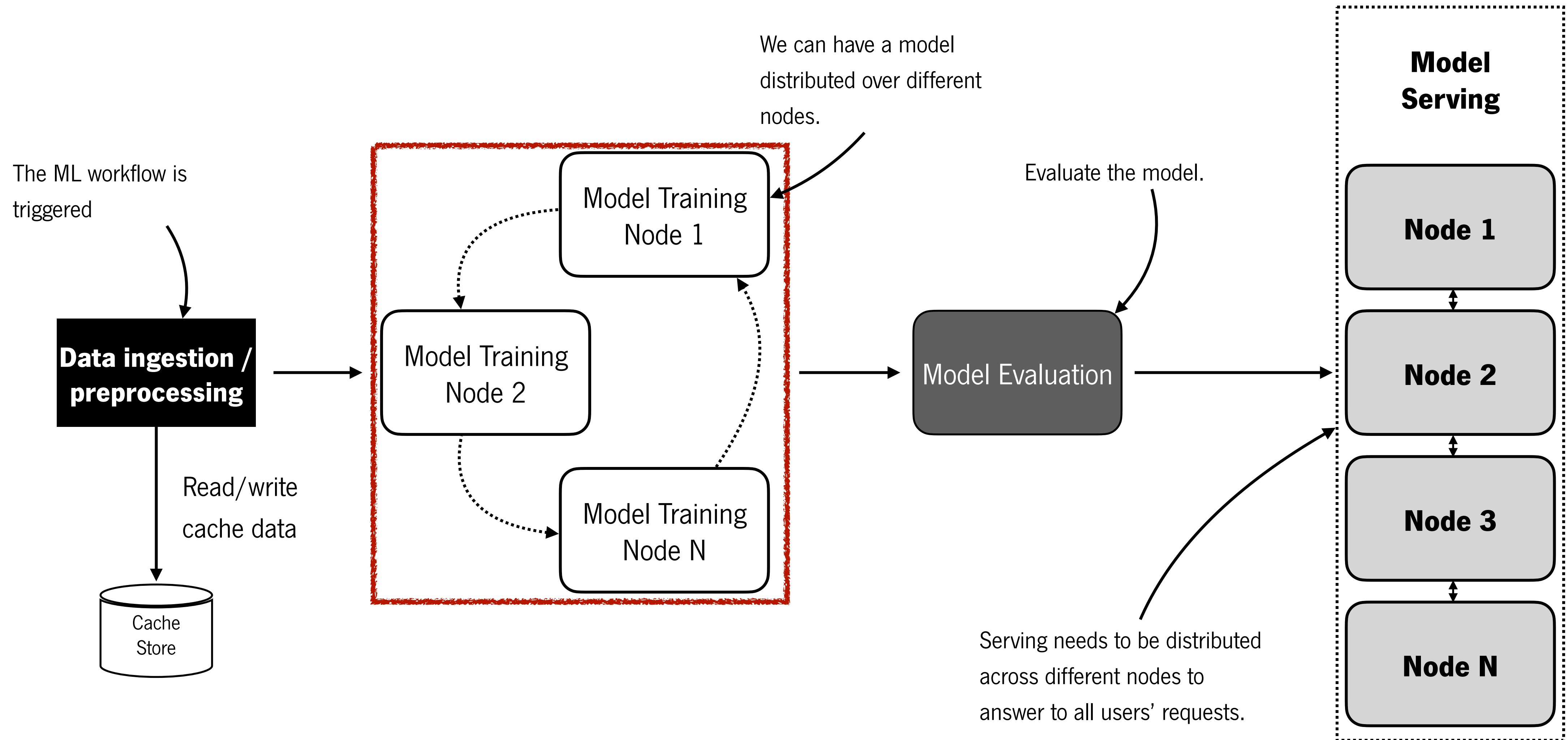
Model Evaluation

**Model
Serving**

From Distributed Training to Distributed Inference



From Distributed Training to Distributed Inference



Distributed Inference vs Distributed Training

- In distributed inference, there is **no backward pass**, but memory and latency constraints are tighter.
- Requests arrive continuously and must be answered in milliseconds.
- The **KV cache** dominates memory, not model weights.
- Scaling means **more replicas** or **smarter batching**, not just more GPUs.

Inference vs Training: A Different Problem

ASPECT	TRAINING	INFERENCE / SERVING
GOAL	Minimise loss	Meet latency + throughput SLAs
COMPUTE	Forward + backward pass	Forward pass only
BATCH SIZE	Large (hundreds–thousands)	1–few (online) or large (offline batch)
MEMORY OCCUPANTS	Weights + activations + gradients + optimiser state	Weights + KV cache (no gradients, no optimiser)
EXECUTION	Long job (hours–weeks)	Real-time, millisecond budgets
BOTTLENECK	Compute-bound (GPU utilisation)	Memory-bandwidth-bound (loading weights each step)
FAILURE MODE	Slow convergence, bad generalisation	SLA violation, OOM crash under load

Two Phases of Autoregressive Generation

Phase 1 — PREFILL

Input: the full prompt (all tokens at once)

- All prompt tokens processed in parallel
- Builds initial KV cache for every prompt token
- Compute-intensive — large matrix multiplications
- Scales quadratically with prompt length ($O(n^2)$)
- Latency here = Time to First Token (TTFT)

Phase 2 — DECODE

Input: one new token at a time

- Generates one token per step
- Reads cached K/V — no recomputation of previous tokens
- Memory-bandwidth-bound — weights loaded every step
- Scales linearly with output length
- Latency here = Time Per Output Token (TPOT)

Memory Breakdown in LLM Inference

Model weights

Fixed. 2 bytes/param in fp16. GPT-3 (175B) $\approx$ 350 GB. Llama-3-8B $\approx$ 16 GB.

KV cache

Dynamic. Grows with context $\times$ batch $\times$ layers. Dominates memory in typical serving scenarios.

Activations

Per-layer intermediates during forward pass. Small at inference — no backprop storage needed.

Framework overhead

CUDA kernels, memory allocator, buffers. Typically 1–3 GB — usually not the bottleneck.

KV cache sizing formula

$$\text{KV_bytes} = 2 \times L \times H \times D \times S \times B \times \text{dtype}$$

L = layers H = attention heads

D = head dim S = seq length

B = batch size $\text{dtype} = 2$ (fp16)

Example: Llama-3-8B

$L=32$, $H=32$, $D=128$, fp16

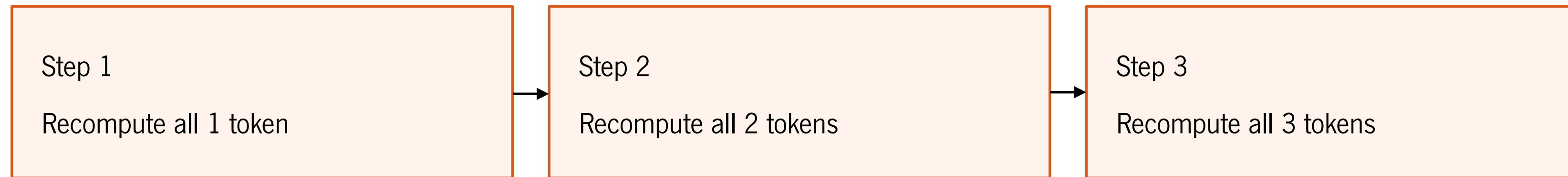
$S=4096$, $B=32$:

$\text{KV} \approx 34 \text{ GB}$ ← bigger than weights!

→ **Model weights: ~16 GB**

→ **KV cache: ~34 GB**

KV Cache and PagedAttention



Cost: $O(n^2)$ — each step recomputes all previous tokens

With KV cache:

```
GPU HBM — KV Cache  
[ K1V1 | K2V2 | K3V3 | ... ] grows each step
```

New token attends to stored K/V — cost: $O(n)$ per step

PagedAttention (vLLM, 2023)

Problem with naive KV cache:

- Contiguous memory block pre-allocated at max context length
- Memory fragmentation: partially-used blocks waste GPU RAM
- Hard to share prompt prefixes across requests

Solution (PagedAttention):

- Store KV blocks in non-contiguous pages — like OS virtual memory
- Allocate pages on demand — no wasted pre-allocation
- Share pages across requests with identical prompt prefix

Batching Strategies: From Static to Continuous

Static Batching

Collect B requests, run together, return all. All must finish before the batch closes.

✓ Simple to implement ✗ GPU idles waiting for the slowest sequence

Dynamic Batching

Batch on timeout or size threshold. Still closes when all sequences in the batch finish.

✓ Lower queuing latency ✗ Padding waste; batch held until all sequences end

Continuous Batching (in-flight batching)

When a sequence emits end-of-sequence (EOS), its slot is immediately filled by the next waiting request. The batch never stops.

✓ Near-optimal GPU utilisation ✗ Requires PagedAttention; more complex scheduler

Hardware Considerations for Inference

Memory bandwidth — not TFLOPS — is the bottleneck in decode

GPU / ACCELERATOR	MEMORY	BANDWIDTH	NOTES
NVIDIA A100 SXM 80GB	80 GB HBM2E	2.0 TB/s	WORKHORSE FOR TRAINING + INFERENCE
NVIDIA H100 SXM 80GB	80 GB HBM3	3.35 TB/s	~1.7× A100 BANDWIDTH; BEST FOR DECODE
NVIDIA H200 141GB	141 GB HBM3E	4.8 TB/s	2× A100 MEMORY + BANDWIDTH
NVIDIA A10G 24GB	24 GB GDDR6	0.6 TB/s	COST-EFFICIENT FOR SMALLER MODELS
GOOGLE TPU v5E	16–32 GB	~1.6 TB/s	DESIGNED FOR INFERENCE AT SCALE

Rule of thumb: arithmetic intensity of LLM decode ≈ 1 FLOP/byte $\rightarrow$ bandwidth-limited, not compute-limited

Roofline model: if ops/byte < hardware ops:byte ratio $\rightarrow$ you are memory-bandwidth-bound

Quantization: Reducing Memory and Increasing Throughput

Represent weights (and optionally activations) in lower precision to reduce memory and increase effective bandwidth.

FORMAT	BITS/PARAM	7B MODEL SIZE	ACCURACY IMPACT	USE CASE
FP32	32	28 GB	NONE (REFERENCE)	TRAINING ONLY
BF16 / FP16	16	14 GB	NEGLIGIBLE	STANDARD INFERENCE BASELINE
FP8	8	7 GB	MINIMAL (< 1% ON MOST TASKS)	H100; GOOD DEFAULT
INT8 (W8A8)	8	7 GB	SMALL (TASK-DEPENDENT)	WIDELY SUPPORTED, MATURE
INT4 (GPTQ/AWQ)	4	3.5 GB	MODERATE — NEEDS CALIBRATION	MEMORY-CONSTRAINED
INT2–3 (EXTREME)	2–3	~2 GB	SIGNIFICANT	RESEARCH / EDGE ONLY

Try quantisation before adding more GPUs — usually the highest-ROI first step.

FlashAttention: Memory-Efficient Attention

Standard Attention — the problem

- Attention matrix $S = QK^T$ has shape $(\text{seq_len} \times \text{seq_len})$
- For $\text{seq_len} = 4096$: $4096 \times 4096 = 16\text{M}$ values per layer
- Must materialise in HBM before softmax
- Memory: $O(n^2)$ — grows quadratically
- 4+ HBM read/write operations per attention op
- HBM bandwidth becomes the bottleneck

FlashAttention — the solution

- Tiles Q, K, V into blocks that fit in SRAM (on-chip, fast)
- Computes attention incrementally — never materialises $n \times n$ matrix
- One pass through HBM only — 5-10× fewer memory ops
- Same exact mathematical result as standard attention
- Memory: $O(n)$ instead of $O(n^2)$
- FlashAttention-2 and -3 push further on H100

Speculative Decoding: Faster Generation

Problem: decode generates one token at a time — the GPU is underutilised per step.

1

Draft model generates K tokens

A small, fast model (e.g. 70M params) autoregressively generates K candidate tokens from the current context. Cost: cheap.

2

Target model verifies all K tokens in parallel

The large model runs ONE forward pass on all K draft tokens simultaneously. Accepts tokens that match its distribution, rejects from the first mismatch.

3

Accepted tokens emitted, generation continues

If all K accepted: K tokens generated in ~ 1 large-model step. If 0 accepted: fall back to 1 token. Expected $\approx 2-4$ tokens accepted in practice.

Speedup: 2–3× on coding/text tasks. Target model output is mathematically identical to standard decoding — pure speed optimisation.

Parallelism Strategies at Inference Time

Tensor Parallelism

When:

Model too large for 1 GPU

How:

Split weight matrices across N GPUs — each holds a column-shard. AllReduce after every attention and MLP layer to combine partial results.

Comm:

AllReduce (NVLink). Keep within one node — cross-node adds ~ms latency per layer.

Frameworks:

vLLM, TensorRT-LLM, Megatron-LM · TP=2/4/8

Pipeline Parallelism

When:

Extremely deep models (many layers)

How:

Assign consecutive layer groups to different devices. Micro-batches flow stage by stage through the pipeline.

Comm:

Point-to-point activations. Adds pipeline bubble latency — less used at inference than training.

Frameworks:

DeepSpeed, FasterTransformer

Expert Parallelism

When:

Mixture-of-Experts models (Mixtral, DeepSeek, GPT-4)

How:

Distribute experts across GPUs. A learned Router sends each token to its top-K experts, only those GPUs activate. Sparse = fewer FLOPs, more parameters.

Comm:

All-to-All (tokens routed to expert GPUs). Load imbalance if most tokens pick the same expert.

Frameworks:

Megablocks, vLLM (MoE), DeepSpeed-MoE

Replica Parallelism

When:

High request volume, model fits on 1 node

How:

Run N fully independent model copies behind a load balancer. Each replica is completely decoupled — no cross-replica communication.

Comm:

None — zero overhead. Linear throughput scaling with replica count.

Frameworks:

Kubernetes, Ray Serve, Triton

Metrics That Matter

TTFT

Time to First Token

Time from request arrival to first output token. Measures perceived responsiveness in interactive applications.

→ *Dominated by prefill phase. Affected by prompt length and server queue depth.*

TPOT

Time Per Output Token

Average time between consecutive output tokens after the first. Determines streaming fluency.

→ *Dominated by decode step. Affected by batch size, KV cache pressure, and memory bandwidth.*

Throughput

Tokens/sec or Requests/sec

Total tokens generated per second across all concurrent requests. Primary metric for batch / offline workloads.

→ *Improved by larger batches and continuous batching. Does NOT account for SLA violations.*

Goodput

SLA-compliant Requests/sec

Requests completed per second that also satisfied their latency SLA. Throughput that actually counts — unhelpful responses ignored.

→ *Goodput $\leq$ Throughput always. A system with high throughput but poor latency can have near-zero goodput.*

MFU

Model FLOPs Utilisation

Observed throughput $\div$ theoretical peak GPU throughput. Measures hardware efficiency end-to-end.

→ *60–80% is excellent for training. Inference decode is memory-bound so lower MFU is expected and normal.*

Serving Frameworks: Current Landscape

vLLM

UC Berkeley / OSS

Use: First choice for LLM serving in research and production

PagedAttention + continuous batching. Widest model support. Best open-source default.

Ray Serve

Anyscale

Use: Multi-model systems; production orchestration

Framework-agnostic serving layer. Auto-scaling, request routing, multi-model. Orchestrates vLLM/TRT-LLM backends.

TGI

Hugging Face

Use: HuggingFace ecosystem; fast cloud deployment

Text Generation Inference. Flash Attention, tensor parallelism. Docker-ready. Powers HuggingFace Inference API.

Triton Server

NVIDIA

Use: Heterogeneous model types; enterprise pipelines

Supports PyTorch, TF, ONNX, TensorRT. Dynamic batching, model ensembles. Enterprise MLOps.

TensorRT-LLM

NVIDIA

Use: Maximum performance on NVIDIA hardware

NVIDIA-optimised kernels, FP8, in-flight batching. Highest throughput on A100/H100. Requires compilation.

Decision Framework: When to Distribute and How

SITUATION	RECOMMENDED APPROACH	EXAMPLE
MODEL FITS ON 1 GPU LOW TRAFFIC	SINGLE GPU. QUANTISE BEFORE ADDING HARDWARE.	<i>LLAMA-3-8B ON 1× A100</i>
MODEL FITS ON 1 GPU HIGH TRAFFIC	REPLICATED DATA PARALLELISM. N REPLICAS BEHIND LOAD BALANCER.	<i>8× A100, 8 REPLICAS OF LLAMA-3-8B</i>
MODEL TOO LARGE FOR 1 GPU MODERATE TRAFFIC	TENSOR PARALLELISM WITHIN A NODE. 2–8 GPUS.	<i>LLAMA-3-70B ON 2× A100 (TP=2)</i>
MODEL TOO LARGE HIGH TRAFFIC	TENSOR PARALLEL WITHIN NODE + REPLICAS ACROSS NODES.	<i>TP=4 × 16 REPLICAS = 64 GPUS</i>
COST-SENSITIVE / EDGE	QUANTISE AGGRESSIVELY (INT4). SINGLE OR FEW GPUS.	<i>LLAMA-3-8B INT4 ON 1× RTX 4090</i>
VERY LONG CONTEXT	QUANTISE KV CACHE (FP8). PAGEDATTENTION, SLIDING WINDOW.	<i>128K CONTEXT ON H100 CLUSTER</i>

Production Serving Architecture

Client / Application

Web, mobile, API consumers

API Gateway / Load Balancer

Auth, rate limiting, routing — Nginx, Envoy, AWS ALB

Serving Orchestration Layer

Auto-scaling, health checks, A/B testing, queuing — Ray Serve, KServe, Triton

Model Inference Workers

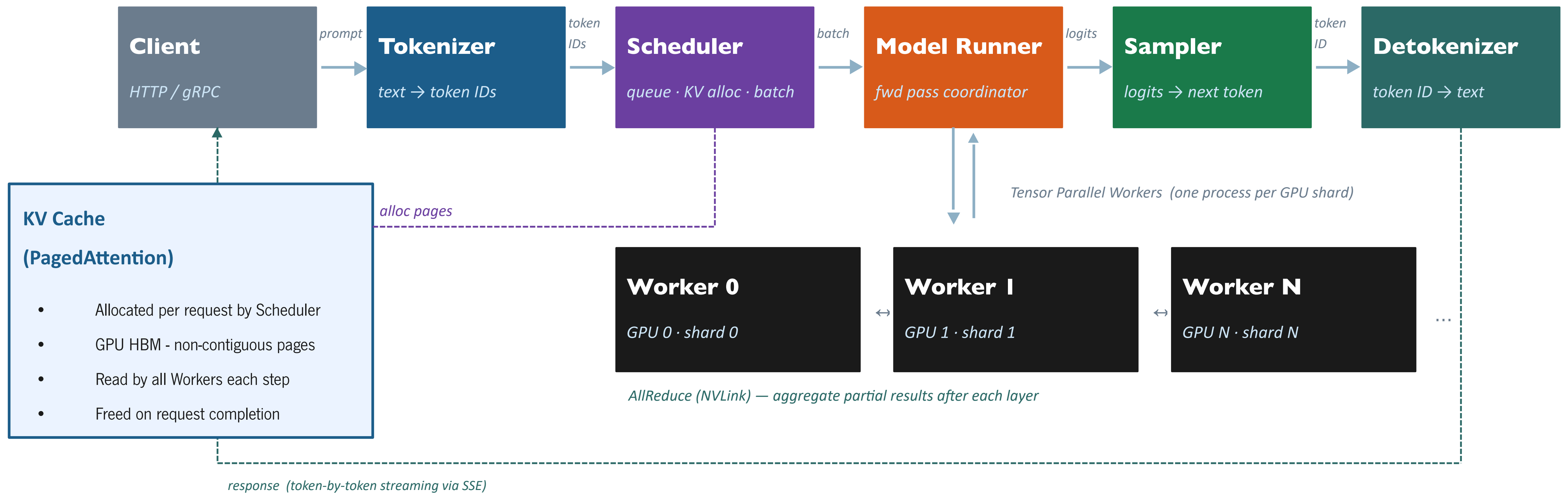
vLLM / TensorRT-LLM — tensor parallelism per replica (1–8 GPUs each)

Observability

TTFT/TPOT P50/P99, GPU utilisation, KV cache %, queue depth — Prometheus, Grafana

Key: statelessness (horizontal scaling), sticky sessions (KV prefix reuse), predictive autoscaling under traffic spikes.

From Request to Response: Inside the Serving Engine



Challenges in Distributed Inference

Memory Fragmentation

KV cache and request sizes vary unpredictably. Naive allocations waste GPU RAM, limiting concurrency. Solution: PagedAttention, KV cache quantisation.

Straggler Requests

One very long request holds back the batch in static batching. Solution: continuous batching with per-sequence scheduling; request timeout policies.

Cold Start Latency

Loading model weights from storage takes seconds to minutes. Solution: warm replicas, memory-mapped files, NVMe SSDs close to GPU.

Heterogeneous Traffic

Prompt and output lengths are unpredictable. Affects KV cache sizing and batching efficiency. Solution: length-aware scheduling, bucketing.

Model Updates / Versioning

Rolling out a new model without downtime. Solution: shadow testing, gradual traffic shifting.

Cost vs Latency Trade-off

GPUs are expensive when idle; scaling down is slow. Solution: autoscaling with prediction, spot instances for offline batch workloads.

Inference Optimisation: Where to Start

Ordered by typical impact-to-effort ratio — always measure before and after each step.

1	Profile and measure first Identify bottleneck: memory-bound, compute-bound, or I/O-bound? Use NVIDIA Nsight, vLLM logs, or simple timing. Without this, optimisation is guesswork.	2	Enable FlashAttention Free 2–4× speedup on attention. Already default in most frameworks. Verify with <code>attn_implementation='flash_attention_2'</code> .
3	Quantise the model INT8 or FP8 for minimal accuracy loss. INT4 (GPTQ/AWQ) if memory is the constraint. Often 1.5–2× throughput and 2–4× memory reduction.	4	Switch to continuous batching Use vLLM or TGI instead of naive HuggingFace generate(). Alone this can give 5–20× throughput improvement for concurrent users.
5	Tune batch size and parallelism Larger batches improve GPU utilisation. Add tensor parallelism when model doesn't fit. Add replicas when throughput is the goal.	6	KV cache optimisation Quantise KV cache to fp8. Enable prefix caching for repeated system prompts. Tune <code>max_model_len</code> to avoid wasted pre-allocation.

Distributed Training vs Distributed Inference:

Summary

DIMENSION	DISTRIBUTED TRAINING	DISTRIBUTED INFERENCE
PRIMARY METRIC	TIME TO ACCURACY (WALL-CLOCK TRAINING TIME)	THROUGHPUT (TOK/S) AND LATENCY (TTFT, TPOT)
MEMORY CONSUMERS	WEIGHTS + ACTIVATIONS + GRADIENTS + OPTIMISER STATE	WEIGHTS + KV CACHE (NO GRADIENTS/OPTIMISER)
COMMUNICATION	ALLREDUCE OF GRADIENTS PER STEP (GB-SCALE)	ALLREDUCE PER LAYER DURING DECODE (MB-SCALE)
BOTTLENECK	COMPUTE-BOUND (LARGE MATMUL BATCHES)	MEMORY-BANDWIDTH-BOUND (DECODE, SMALL BATCH)
SCALING STRATEGIES	DATA / MODEL / PIPELINE PARALLELISM	REPLICAS + TENSOR PARALLEL + CONTINUOUS BATCHING
KEY TECHNIQUES	MIXED PRECISION, GRADIENT CHECKPOINTING, ZERO	QUANTIZATION, FLASHATTENTION, PAGEDATTENTION, SPECULATIVE DECODING
FRAMEWORKS	DDP, HOROVOD, DEEPSPEED, MEGATRON-LM	vLLM, TGI, TENSORRT-LLM, TRITON, RAY SERVE

Edge and Federated Inference

Not all inference happens in a data centre.

On-device inference

Run models on user devices (phones, laptops). Requires INT4/INT2, small model families (Gemma 2B, Phi-3, Mistral 7B). No network latency, privacy by default. Frameworks: llama.cpp, Apple MLX, ExecuTorch.

e.g. Apple Intelligence, Google Pixel AI

Edge serving

Run models on edge nodes close to users. Reduces round-trip latency vs cloud. Lower hardware (A10G, L4, consumer GPUs). Same frameworks as cloud but smaller models.

e.g. CDN-attached GPU nodes, retail/industrial AI

Split inference

Split the model across device + cloud. Early layers on-device (fast, cheap), later layers in cloud. Reduces data sent to cloud. Active research area.

e.g. Neurosymbolic / hybrid AI pipelines

Privacy considerations

Sending prompts to a cloud LLM leaks user data. On-device inference or self-hosted serving are increasingly demanded. Regulation (GDPR, HIPAA) affects deployment architecture.

e.g. Healthcare, legal, finance verticals

Takeaways

01

Inference $\neq$ Training

Different bottleneck (memory-bandwidth), different memory (KV cache), different metrics (TTFT/TPOT). A different set of engineering challenges.

02

KV cache is central

Grows with context $\times$ batch $\times$ layers. PagedAttention manages it like virtual memory. Quantise it (fp8) for longer contexts.

03

Continuous batching is the baseline

Use vLLM or TGI. Static batching wastes GPU time. Continuous batching keeps the GPU busy — 5–20 $\times$ throughput improvement.

04

Quantise before you scale

INT8/FP8 is nearly free in accuracy. INT4 halves memory. Always try this before adding more hardware.

05

Distribute in the right dimension

Replicas for throughput (no overhead). Tensor parallelism for memory (adds AllReduce). Measure first.

06

Measure the right metric

TTFT for interactive apps. Throughput for batch. MFU to diagnose hardware efficiency. Profile before optimising.

Further Reading

KV Cache & PagedAttention

- Kwon et al. (2023) — Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023.
- vLLM documentation — <https://docs.vllm.ai>

FlashAttention

- Dao et al. (2022) — FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS 2022.
- Dao (2023) — FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning.

Continuous Batching

- Yu et al. (2022) — Orca: A Distributed Serving System for Transformer-Based Generative Models. OSDI 2022.

Speculative Decoding

- Leviathan et al. (2023) — Fast Inference from Transformers via Speculative Decoding. ICML 2023.

Quantisation

- Dettmers et al. (2022) — LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale.
- Frantar et al. (2022) — GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers.

Benchmarks

- MLCommons MLPerf Inference — <https://mlcommons.org/benchmarks/inference-datacenter/>

Questions?